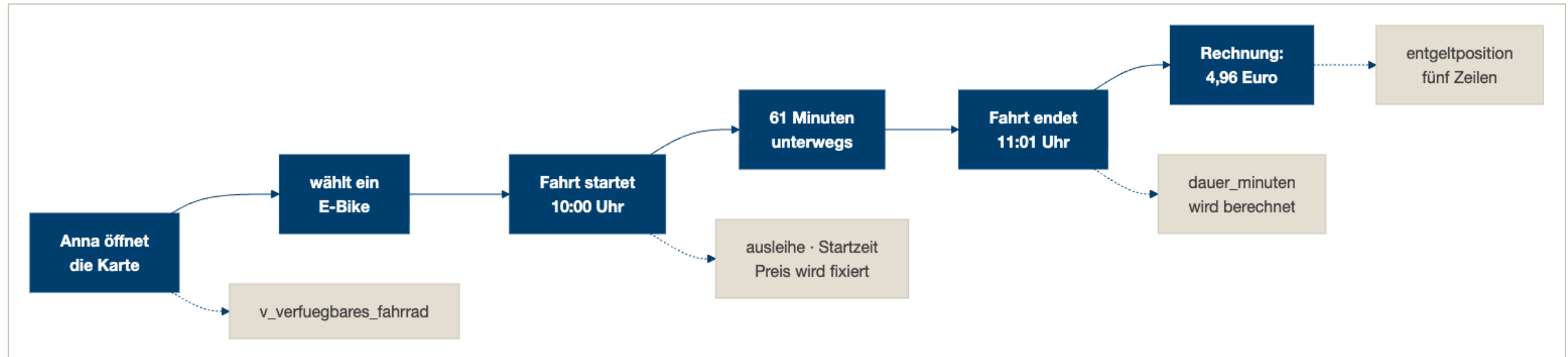


Datenbankentwurf am Fallbeispiel VeloCity

Eine Fahrt, 61 Minuten, 4,96 Euro — und alles

Anna fährt 61 Minuten. Das ist alles, was wir wissen.

Aus diesem einen Satz entsteht das gesamte Datenmodell. Jede Frage, die diese Fahrt aufwirft, beantwortet ein Kapitel dieser Einheit — und am Ende steht eine Datenbank, die Annas Rechnung selbst garantiert.



Prüfen Sie im Verlauf mit: Woher weiß die Datenbank am Ende, dass Anna genau 4,96 Euro zahlt — und nicht 6,20 oder 4,00?

Neun Kapitel, neun Fragen an Annas Fahrt

Die Reihenfolge ist zwingend. Anforderungen vor Modell, Modell vor Relationen, Relationen vor DDL. Wer sie umdreht, bindet sich an eine Umsetzung, bevor er weiß, was umzusetzen ist.

Kapitel	Die Frage, die Annas Fahrt aufwirft
1 Fallstudie	Was muss die Datenbank über diese Fahrt überhaupt wissen?
2 Konzeptionell	Welche Dinge sind das — und wie hängen sie zusammen?
3 Normalisierung	Warum reicht dafür nicht eine einzige Tabelle?
4 Logisch	Wie wird aus dem Bild ein Satz von Relationen?
5 Physisch	Wie erzwingt die Datenbank, dass Annas Rechnung stimmt?
6 Implementierung	Was kostet die Fahrt genau — und warum?
7 Sicherheit	Wer außer Anna darf ihre Daten sehen?
8 Anbindung	Wie kommt das alles in die Web-Anwendung?
9 Ausblick	Was fehlt noch, damit die Räder gewartet werden?

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter [doku/datenmodell/](https://github.com/butscher/datenmodell/).

1 · Die Fallstudie

Leitfrage

Was muss die Datenbank über Annas Fahrt überhaupt wissen?

VeloCity vermietet Räder minutengenau

Ein Bike-Sharing-System in Würzburg. Kunden finden freie Räder auf einer Karte, entleihen sie, fahren und stellen sie wieder ab. Vielfahrer schließen einen Tarif ab, der Freiminuten und Rabatt mitbringt.

Drei Fahrzeugklassen

City-Bike, Stadtrad ohne Motor
E-Bike Sport, Pedelec bis 25 km/h
E-Cargo Loader, bis 100 kg Zuladung

Zwei Abstellarten

An einer festen Station, kostenfrei
Frei im Stadtgebiet gegen Zuschlag
Beides muss das Modell abbilden

Vier Tarife

Basis ohne Freiminuten
Student und ÖPNV mit Kontingent
Premium mit Rabatt und Monatspreis

Annas Fahrt: Anna fährt ein E-Bike und hat den Studententarif — 300 Freiminuten im Monat.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Zehn Geschäftsregeln — jede muss erzwungen werden

Diese Regeln sind die fachliche Substanz. Entscheidend ist nicht, sie zu notieren, sondern dass jede einzelne am Ende erzwungen wird — von der Datenbank, nicht von der Anwendung.

1	GR1 Ein Rad ist höchstens einmal gleichzeitig ausgeliehen	partieller UNIQUE-Index
2	GR2 Ein Kunde hat höchstens vier aktive Ausleihen	Prüfung in der Funktion
3	GR3 Ein Kunde hat zu einem Zeitpunkt einen Tarif	EXCLUDE-Constraint
4	GR4 Preise und Konditionen überlappen sich nie	EXCLUDE-Constraint
5	GR5 Bepreist wird mit dem Preis zur Startzeit	in fn_ausleihe_beenden
6	GR6 Angefangene Minuten werden aufgerundet	GENERATED-Spalte
Annas Fahrt: GR5 entscheidet über Annas Rechnung: es gilt der Preis um 10:00 Uhr, nicht der von heute.		

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

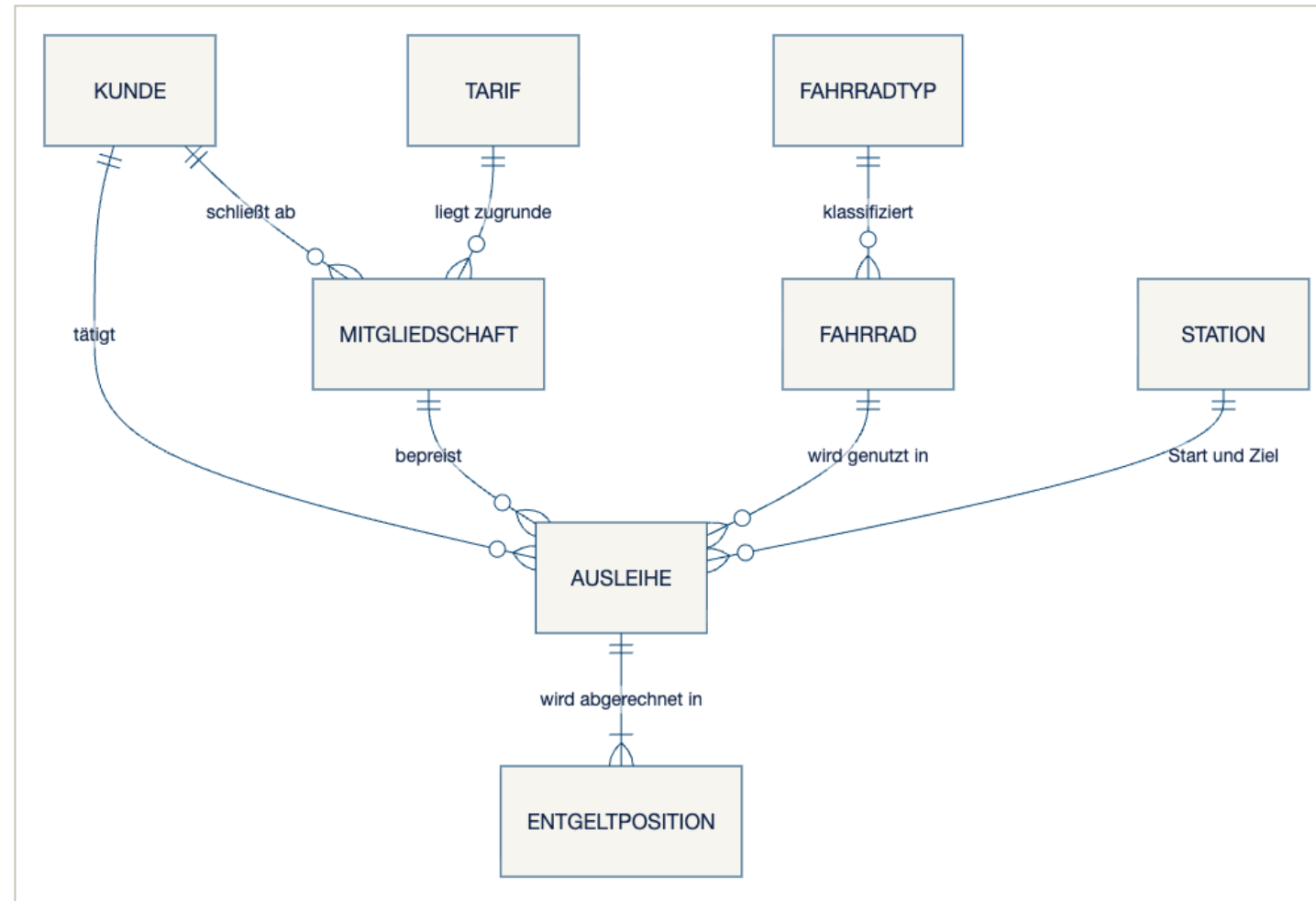
2 · Konzeptioneller Entwurf

Leitfrage

Welche Dinge sind das — und wie hängen sie zusammen?

Substantive werden Entitäten, Verben werden Beziehungen

Eine Heuristik zum Einstieg, kein Automatismus — die eigentliche Arbeit steckt in den Kardinalitäten.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Kardinalitäten lesen und prüfen

Die Notation ist schnell gelernt. Die Frage dahinter ist die schwierige: Stimmt die Aussage über die Wirklichkeit?

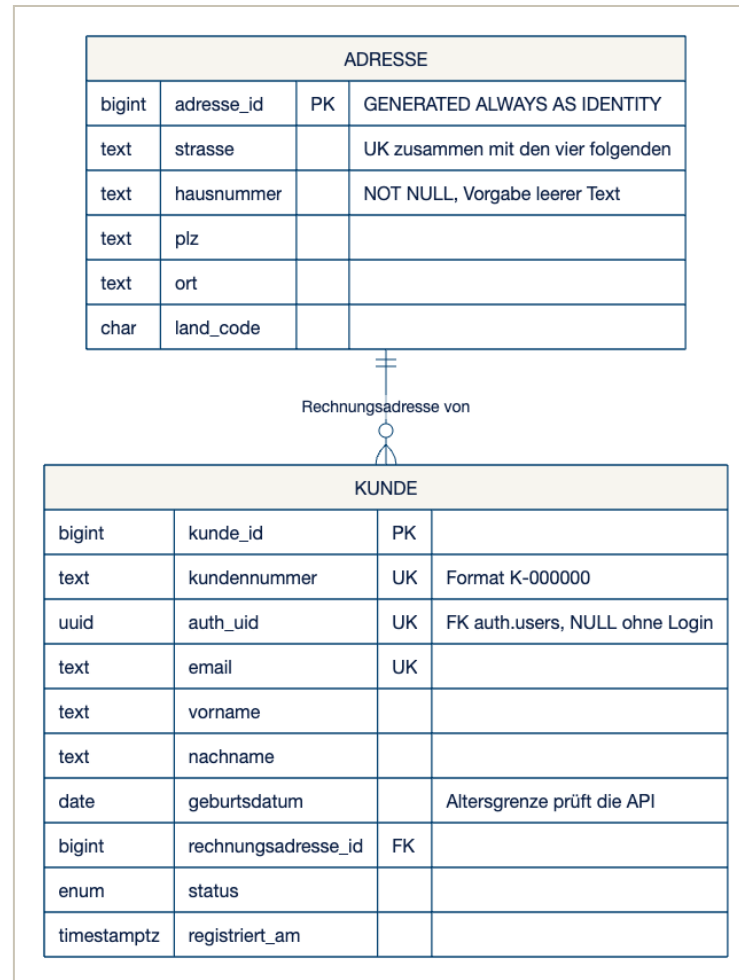
Zeichen	Bedeutung	Beispiel im Modell	Ist das wirklich so?
--o{	eins zu null oder viele	KUNDE tätigt AUSLEIHE	Neukunde ohne Fahrt: ja
-- {	eins zu ein oder viele	RECHNUNG hat POSITION	Rechnung ohne Posten wäre sinnlos
--	eins zu eins	FAHRRAD hat POSITION	genau eine Position je Rad
--o	eins zu null oder eins	AUSLEIHE zu POSITION	noch nicht fakturiert: ja

Annas Fahrt: Annas Fahrt ist eine AUSLEIHE — sie verbindet Kunde, Rad, Station und Tarif in einem Vorgang.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Geschäftspartner: Adresse als eigene Entität

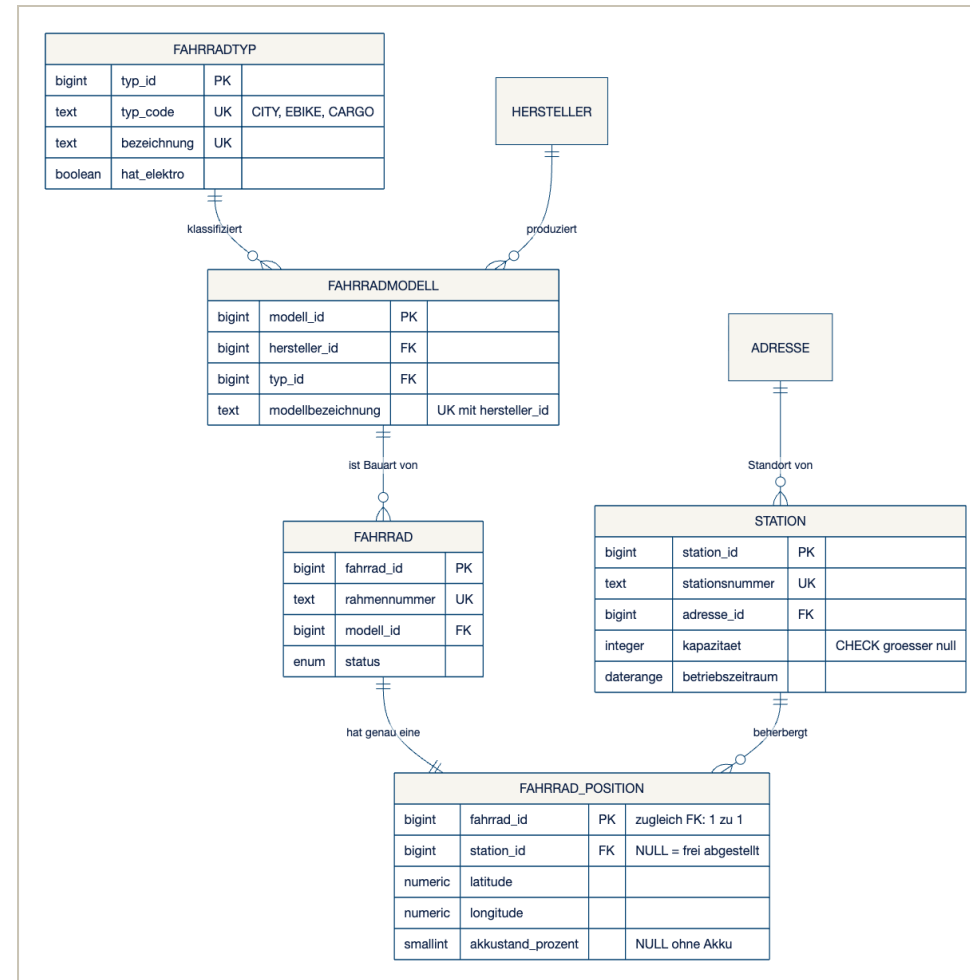
Annas Fahrt: Anna ist ein Satz in kunde — mit Kundennummer, aber ohne Passwort.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Netz und Flotte: Stammdaten und Bewegungsdaten getrennt

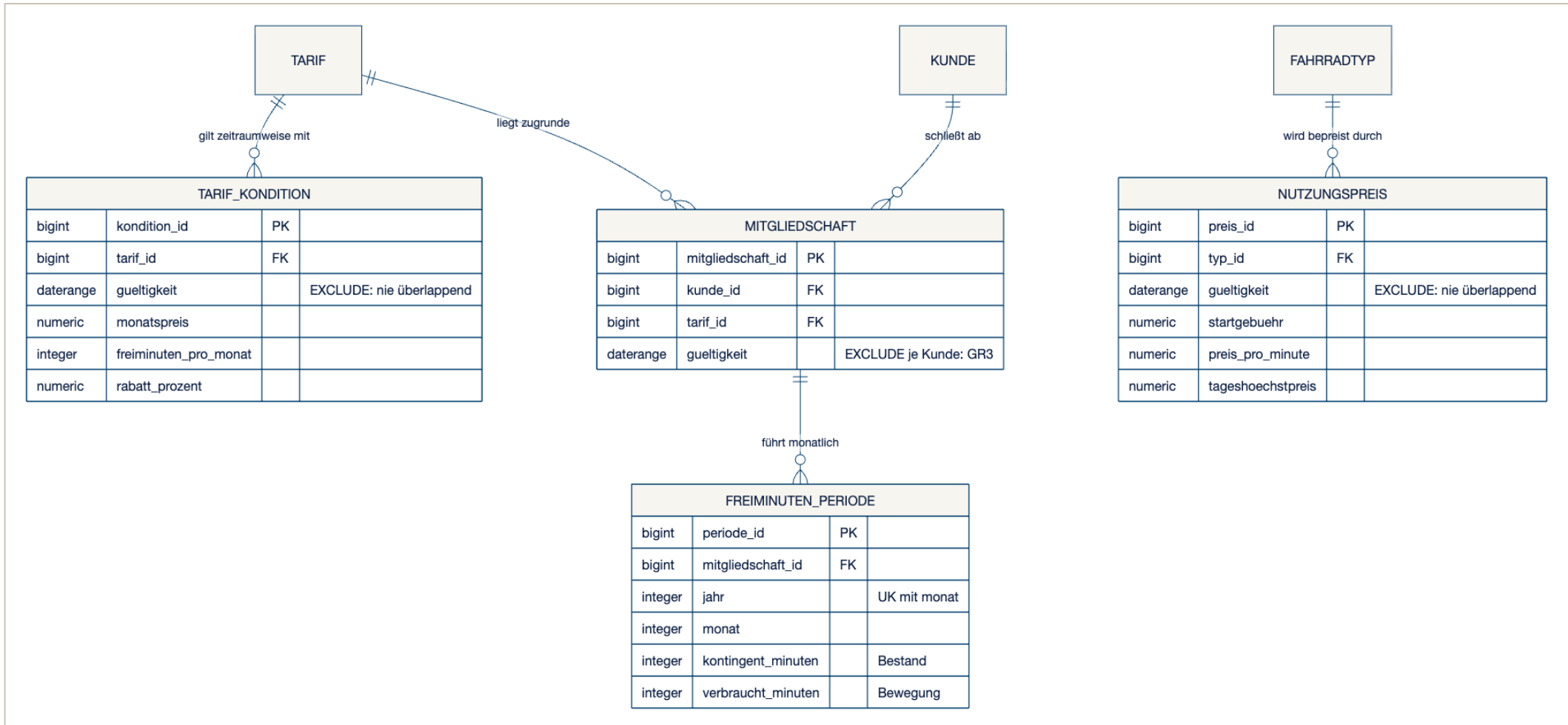
Annas Fahrt: Annas E-Bike steht vor der Fahrt an einer Station, danach vielleicht frei im Stadtgebiet.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Tarif und Preis: alles ist zeitabhängig

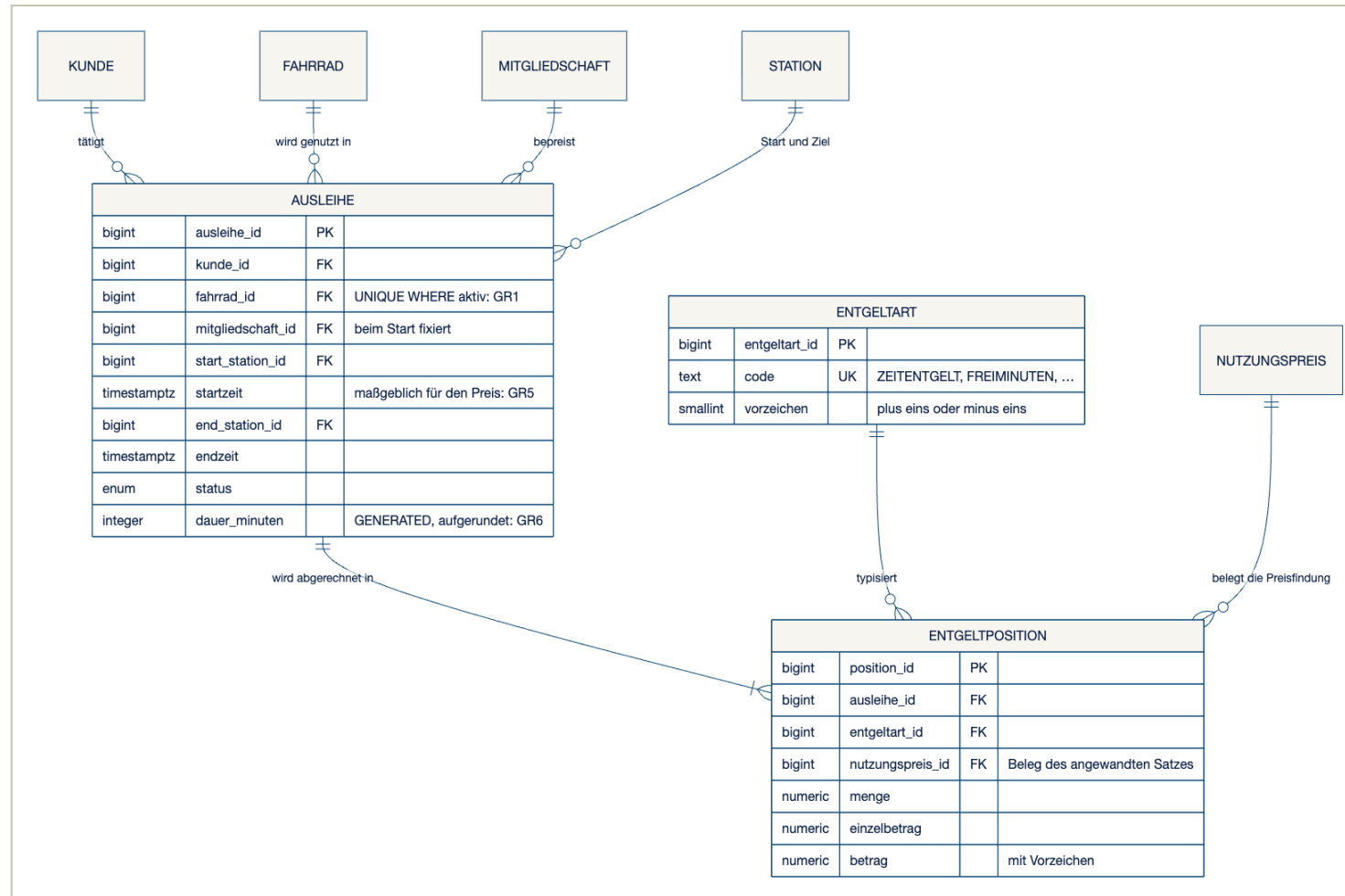
Annas Fahrt: Annas Studententarif bringt 300 Freiminuten im Monat und wird beim Start ihrer Fahrt fixiert.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Nutzung: die Ausleihe und ihre Abrechnung

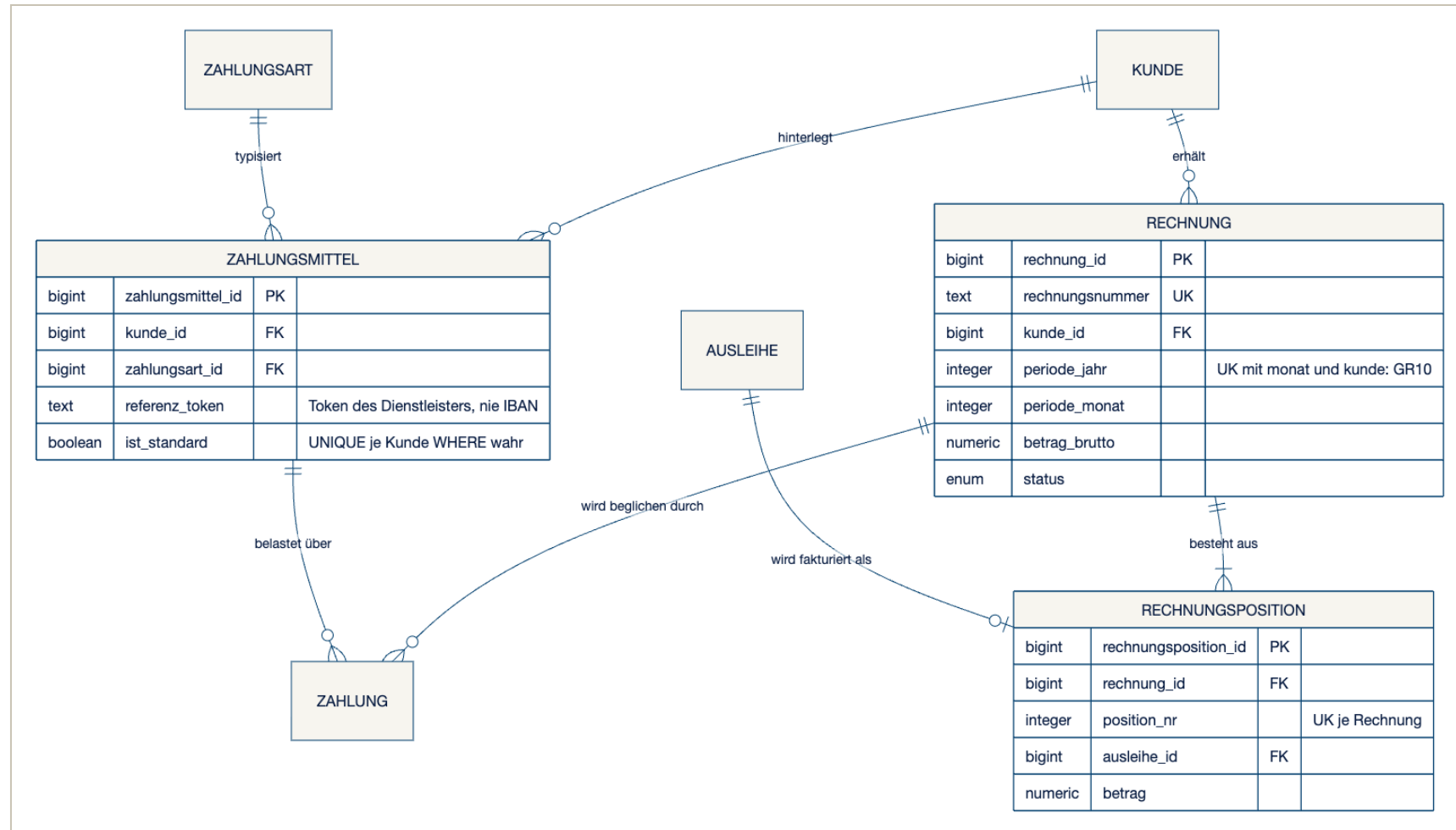
Annas Fahrt: Annas Fahrt erzeugt fünf Entgeltpositionen — jede einzeln nachvollziehbar.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Abrechnung: Beleg, Positionen, Zahlung

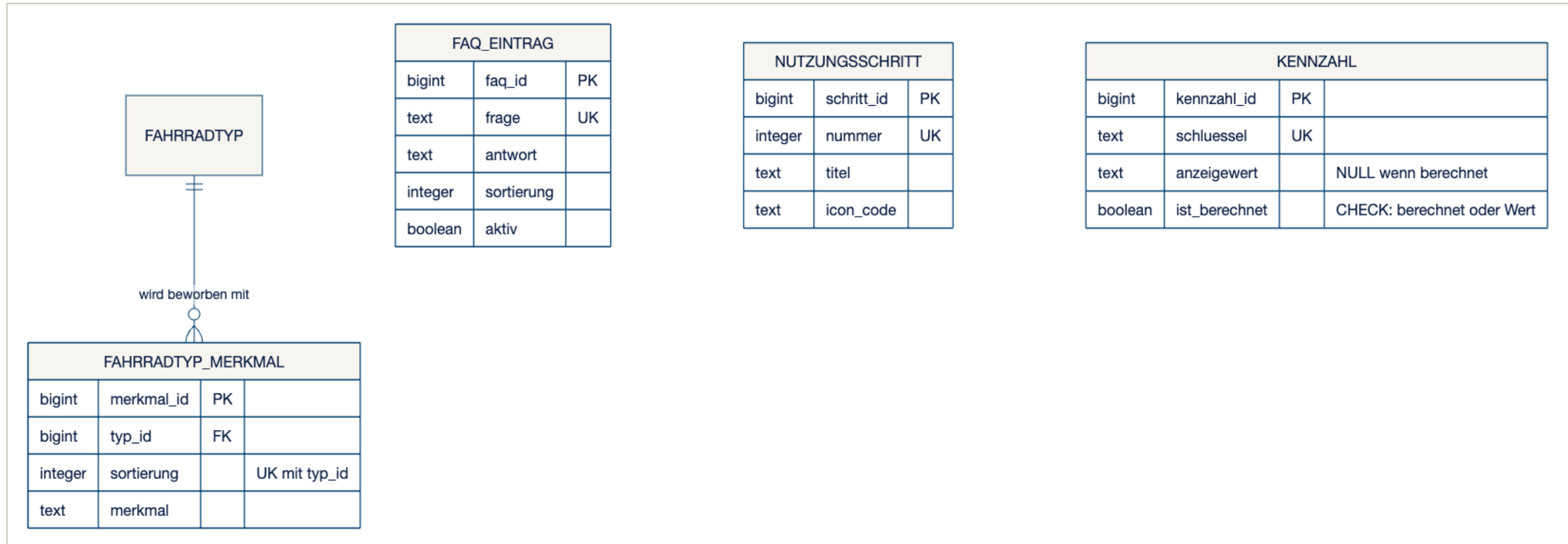
Annas Fahrt: Annas Fahrt erscheint am Monatsende als eine Zeile auf einer Rechnung.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Redaktionsinhalte: warum drei Tabellen und nicht eine

Annas Fahrt: Der Preis, den Anna auf der Tarfkarte sieht, wird aus nutzungspreis gerechnet — nicht getippt.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

3 · Normalisierung

Leitfrage

Warum reicht für Annas Fahrt nicht eine einzige Tabelle?

Der naive Versuch: alles in eine Tabelle

So würde jemand Annas Fahrt aufschreiben, der noch nicht normalisiert hat. Die Tabelle funktioniert — bis zur ersten Änderung.

AUSLEIHE_FLACH			
bigint	ausleihe_id	PK	
text	kunde_email		wiederholt sich je Fahrt
text	kunde_name		nicht atomar: 1NF
text	kunde_strasse		wiederholt sich
text	kunde_plz		
text	kunde_ort		
text	rahmennummer		
text	typ_bezeichnung		hängt am Rad: 3NF
numeric	startgebuehr		hängt am Typ: 3NF
numeric	preis_pro_minute		hängt am Typ: 3NF
timestampz	startzeit		
timestampz	endzeit		
integer	dauer_minuten		abgeleitet
numeric	kosten		abgeleitet
text	positionen		mehrwertig: 1NF

Drei Anomalien, die daraus folgen

Einfügen: ein neuer Fahrradtyp lässt sich ohne Ausleihe gar nicht anlegen.

Ändern: eine Preisanpassung müsste in jeder einzelnen Ausleihzeile nachgezogen werden.

Löschen: mit der letzten Ausleihe eines Typs verschwindet auch dessen Preis.

1NF und 2NF: atomar, und alles hängt am ganzen Schlüssel

Die 1NF verlangt atomare Attribute. Der lehrreichere Fall ist die wiederholende Gruppe: sie wird zu einer eigenen Relation, nicht zu mehreren Spalten.

verletzt

1NF · nicht atomar

kunde_name = „Anna Beispiel“

positionen = „Start 0,10; Zeit 6,10; ...“

Falsche Reparatur:

position1, position2, position3

→ legt die Anzahl im Schema fest

erfüllt

1NF · zerlegt

vorname = „Anna“

nachname = „Beispiel“

entgeltposition als eigene Relation,

eine Zeile je Preisbestandteil

→ die Anzahl ist offen

2NF: bei zusammengesetztem Schlüssel (ausleihe_id, position_nr) hängt kunde_email nur an ausleihe_id — daraus folgt die Trennung in Kopf und Positionen.

3NF — und warum sie hier nicht genügt

Die transitive Kette vom Rad über den Typ zum Preis ist der klassische 3NF-Verstoß. Sie aufzulösen ist notwendig, reicht aber fachlich nicht aus.

3NF verletzt

Die transitive Kette

ausleihe_id

→ rahmennummer

→ typ_bezeichnung

→ preis_pro_minute

Eine Preisänderung müsste in jeder Ausleihzeile nachgezogen werden.

3NF erfüllt — reicht trotzdem nicht

Zerlegt, aber immer noch falsch

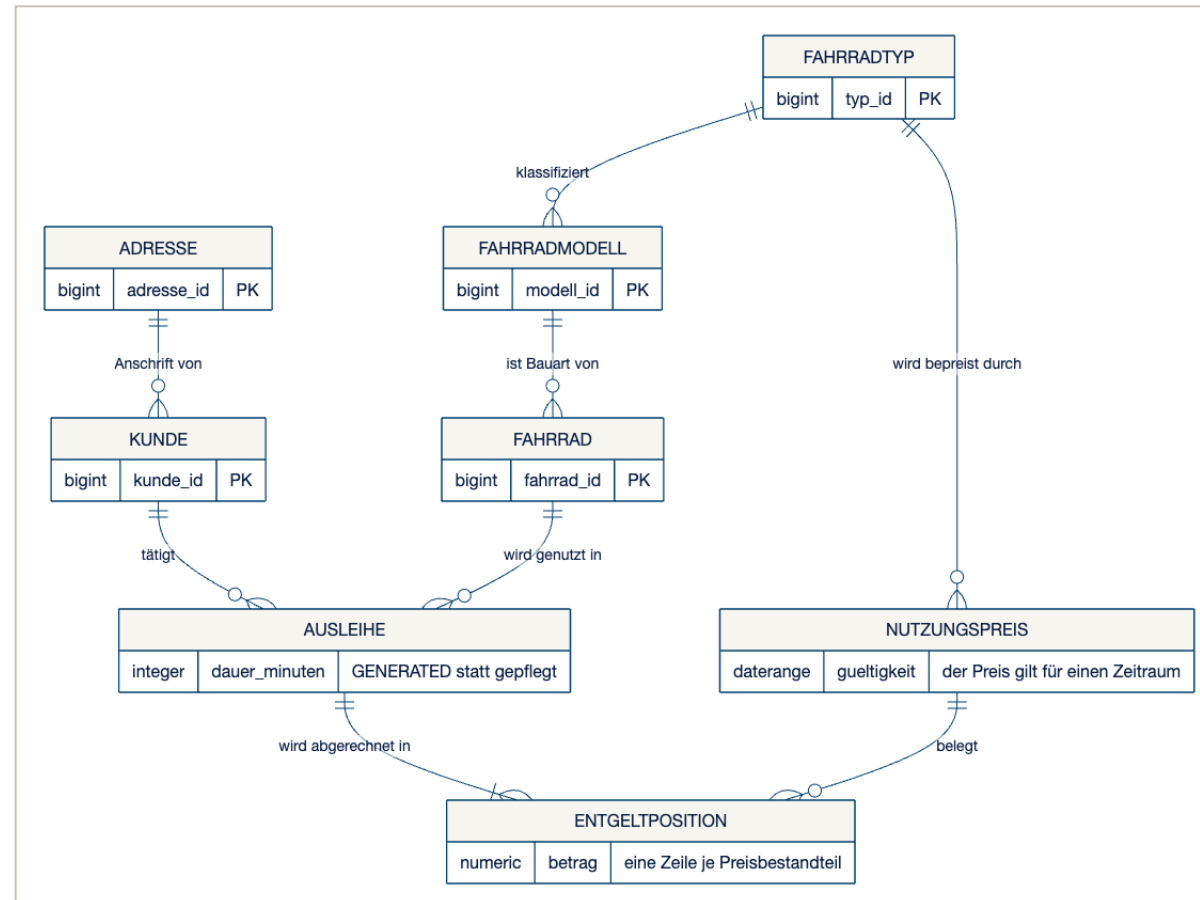
fahrradtyp und nutzungspreis werden eigene Relationen.

ABER: liegt der Preis am Typ, ändert eine Anpassung weiterhin rückwirkend ALLE Altrechnungen — auch Annas.

Erst der Gültigkeitszeitraum in nutzungspreis macht das Modell fachlich richtig. Normalisierung beseitigt Redundanz — sie ersetzt keine fachliche Analyse.

Das Ergebnis der Zerlegung

Aus einer Tabelle sind acht geworden — jede mit genau einer Aufgabe.



Annas Fahrt: Annas Fahrt liegt jetzt in ausleihe — ihr Preis in nutzungspreis, ihre Abrechnung in entgeltposition.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Exkurs: aus der Postleitzahl folgt nicht der Ort

Der Lehrbuchklassiker lautet, plz bestimme ort, also gehöre ort in eine eigene Relation. In Deutschland stimmt das nicht — und der scheinbar saubere Zerlegungsschritt wäre fachlich falsch.

gilt

Eine PLZ hat genau einen Ort		Ländliche Sammel-PLZ umfassen mehrere Orte
Ein Ort hat genau eine PLZ		Großstädte haben Dutzende
plz → ort ist funktional		Also keine transitive Abhängigkeit, kein Split

Eine funktionale Abhängigkeit ist eine Behauptung über die Wirklichkeit und muss geprüft werden — nicht aus Namensähnlichkeit geschlossen.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

4 · Logischer Entwurf

Leitfrage

Wie wird aus dem Bild ein Satz von Relationen?

Die Abbildung folgt festen Regeln

Dieser Schritt ist mechanisch — und genau deshalb korrigierbar. Ein Fehler hier ist kein Geschmacksurteil, sondern ein Regelverstoß.

ERM-Konstrukt	Abbildung ins Relationenmodell	Beispiel im Modell
Entitätstyp	eigene Relation	kunde, station, ausleihe
Attribut	Spalte	vorname, kapazitaet
1:N-Beziehung	Fremdschlüssel auf der N-Seite	ausleihe.kunde_id
1:1-Beziehung	FK auf der optionalen Seite, dort zugleich PK	fahrrad_position.fahrrad_id
M:N-Beziehung	eigene Verknüpfungsrelation	Phase 2: wartungsposition
Mehrwertiges Attribut	eigene Relation	fahrradtyp_merkmal

Annas Fahrt: Annas Fahrt bekommt vier Fremdschlüssel: Kunde, Rad, Mitgliedschaft, Startstation.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Jede Relation trägt zwei Schlüssel

Surrogatschlüssel als Primärschlüssel, fachlicher Schlüssel als Eindeutigkeitsbedingung. Beide haben eine Aufgabe, keiner ersetzt den anderen.

	stabil	eindeutig	sprechend	
Surrogatschlüssel kunde_id	✓	✓	✗	Primärschlüssel
Fachschlüssel kundenummer	✗	✓	✓	UNIQUE-Bedingung
Fachschlüssel als Primärschlüssel	✗	✓	✓	Änderung zieht durch alle Verweise

Eine Rahmennummer wird beim Rahmentausch neu vergeben. Ein Primärschlüssel, der sich ändert, zieht die Änderung durch jede verweisende Relation.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

5 · Physischer Entwurf

Leitfrage

Wie erzwingt die Datenbank, dass Annas Rechnung stimmt?

Datentypen sind fachliche Entscheidungen

Jede Zeile hat einen Grund, der über Geschmack hinausgeht. Die beiden ersten sind die, an denen in der Praxis am meisten schiefgeht.

Zweck	Typ	Warum genau dieser
Zeitpunkt	timestampz	timestamp ohne Zone verliert den Bezug; Sommerzeit wird doppeldeutig
Geldbetrag	numeric(10,2)	exakte Dezimalarithmetik; float trifft 0,10 nicht genau
Zeitraum	daterange	ein Zeitraum ist ein Wert, kein Spaltenpaar — erst so prüfbar
Surrogatschlüssel	bigint IDENTITY	standardkonform; ALWAYS verhindert Setzen von außen
Koordinate	numeric(9,6)	sechs Nachkommastellen entsprechen etwa elf Zentimetern
Text	text	PostgreSQL speichert text und varchar(n) identisch

Annas Fahrt: Annas Startzeit als timestamp ohne Zone wäre in der Nacht der Zeitumstellung nicht eindeutig.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Sieben von zehn Regeln erzwingt die Datenbank

Die oberen fünf gelten immer, auch bei direktem SQL-Zugriff. Die unteren zwei nur, wenn man den vorgesehenen Weg nimmt — sie brauchen Kontext, den ein Constraint nicht hat.

Regel	Umsetzung	Wirkt
GR1	CREATE UNIQUE INDEX ... WHERE status = 'aktiv'	immer
GR3, GR4	EXCLUDE USING gist (... WITH =, ... WITH &&)	immer
GR6	GENERATED ALWAYS AS (ceil(...)) STORED	immer
GR7	CHECK (verbraucht <= kontingent)	immer
GR10	UNIQUE (kunde_id, periode_jahr, periode_monat)	immer
GR2, GR5	Prüfung in fn_ausleihe_starten und _beenden	nur über die Funktion
GR8, GR9	Prüfung in der api_-Schicht	nur über die Funktion

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

EXCLUDE verhindert, was UNIQUE nicht kann

Ein UNIQUE kennt nur Gleichheit. Für „diese Zeiträume dürfen sich nicht überschneiden“ braucht es einen Operator, der Überlappung prüfen kann.

Der Constraint

```
constraint nutzungspreis_ueberschneidung_ex
exclude using gist (
    typ_id          with =,
    gueltigkeit with &&
)
```

Keine zwei Zeilen mit gleichem typ_id
UND überlappenden Zeiträumen.

Zwei Fallstricke

- 1 Voraussetzung btree_gist
Ohne die Erweiterung fehlt bigint
die Operatorklasse für gist:
„data type bigint has no default
operator class for access method
gist“
- 2 Halboffene Grenzen '['']'
Sonst ist der Wechseltag doppelt
belegt.

Annas Fahrt: Ohne GR4 könnte es für Annas Fahrtzeitpunkt zwei gültige Preise geben — die Rechnung wäre nicht bestimmt.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Warum das Mindestalter kein CHECK sein darf

Naheliegender wäre eine Bedingung mit `current_date`. PostgreSQL akzeptiert sie sogar. Trotzdem ist sie falsch — und der Grund zeigt, was ein CHECK eigentlich ist.

verlockend und falsch

CHECK mit `current_date`

```
check (geburtsdatum <=
      current_date - interval '16 years')
```

`current_date` ist nicht IMMUTABLE.
Ein CHECK wird beim Schreiben geprüft
UND beim Wiedereinspielen eines Dumps.

Ein Kunde, der bei der Anmeldung 16 war,
bleibt es — der Restore bricht trotzdem ab.

richtig

Immutable auf der Tabelle, Regel in der Funktion

```
check (geburtsdatum is null
      or geburtsdatum between
        date '1900-01-01'
        and date '2100-01-01')
```

```
-- die Altersregel in api_profil_aktualisieren:
if p_geburtsdatum >
  current_date - interval '16 years'
then return 'Mindestalter nicht erreicht';
```

Merksatz: Ein CHECK darf nur von der Zeile selbst abhängen — und von nichts, was sich mit der Zeit ändert.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter [doku/datenmodell/](#).

6 · Implementierung

Leitfrage

Was kostet Annas Fahrt genau — und warum?

Zwölf Aufbauschritte, jeder für sich lauffähig

Jede Datei ist idempotent: sie läuft zweimal hintereinander fehlerfrei. Das ist die Voraussetzung dafür, dass man einen Aufbau gefahrlos wiederholen kann.

0001 Schema, Erweiterungen, Aufzählungstypen, Audit-Mechanik

0002 bis 0007 · die sechs Fachbereiche A bis F, 25 Tabellen

0008 Referenzdaten: Entgeltarten, Preise, Tarife, Inhalte

0009 Geschäftslogik: fn_-Fachlogik und api_-Zugriffsschicht

0010 Sichten · 0011 Zugriffsschutz · 0012 Data Dictionary

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Annas Rechnung: 4,96 Euro, Zeile für Zeile

Das Zeitentgelt wird über ALLE Minuten gebildet und die Freiminuten als eigene Gutschrift abgezogen. So ist auf der Rechnung ablesbar, was der Tarifvorteil wert war.

Position	Menge	Einzelbetrag	Betrag	Woher der Wert kommt
STARTGEBUEHR	1	0,10	+ 0,10	nutzungspreis, gültig um 10:00 Uhr
ZEITENTGELT	61	0,10	+ 6,10	dauer_minuten, aufgerundet
FREIMINUTEN	0	0,10	– 0,00	Kontingent im Monat schon verbraucht
TARIFRABATT	1	20 %	– 1,24	tarif_kondition zum Startzeitpunkt
HOECHSTPREIS	1	—	– 0,00	6,20 liegt unter der Obergrenze 15,00

6,20 – 1,24 = 4,96 Euro. Reihenfolge: Rabatt VOR der Kappung — umgekehrt käme 4,00 heraus, weil der Rabatt den bereits gedeckelten Betrag ein zweites Mal senken würde.

Annas Fahrt: Damit ist die Frage vom Anfang beantwortet: 4,96 Euro, und jede Zeile ist belegt.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Dokumentation, die nicht veralten kann

Ein Data Dictionary, das von Hand gepflegt wird, ist nach dem zweiten Release falsch. Die Lösung ist, es aus dem Systemkatalog zu erzeugen — und die Vollständigkeit maschinell zu erzwingen.

Beschreibung an das Objekt

```
comment on column
  velocity.fahrrad_position.station_id is
  'NULL bedeutet: das Rad steht frei
  abgestellt, nicht an einer Station';
```

Der Kommentar erklärt, was NULL bedeutet – nicht, wie die Spalte heißt.

Vollständigkeit erzwingen

```
-- Test schlägt fehl, solange eine
-- Fachspalte ohne Kommentar ist
where col_description(c.oid, a.attnum)
      is null
      and a.attname not in
        ('erstellt_am', 'geändert_am');
```

316 Spalten, 264 beschrieben,
52 technische Audit-Spalten ausgenommen.

7 · Zugriffsschutz

Leitfrage

Wer außer Anna darf Annas Daten sehen?

Der Schlüssel im Browser ist kein Geheimnis

Supabase liefert den anon-Key an jeden Besucher aus. Er steht im Quelltext der Seite. Jeder kann damit beliebige Anfragen stellen — mit curl, ohne die Website je zu öffnen.

das Antipattern

Der häufigste Supabase-Anfängerfehler

```
create policy "alles fuer alle"
on kunde for all
to anon, authenticated
using (true) with check (true);
```

Folge: jeder mit dem öffentlichen Schlüssel liest UND ändert sämtliche Kundendaten — auch Annas.

default deny

Was nicht erlaubt ist, ist verboten

```
-- RLS auf jeder Basistabelle
alter table ... enable row level security;
```

```
-- keine Policy für anon
-- Lesen nur über v_-Sichten
-- Schreiben nur über api_-Funktionen
```

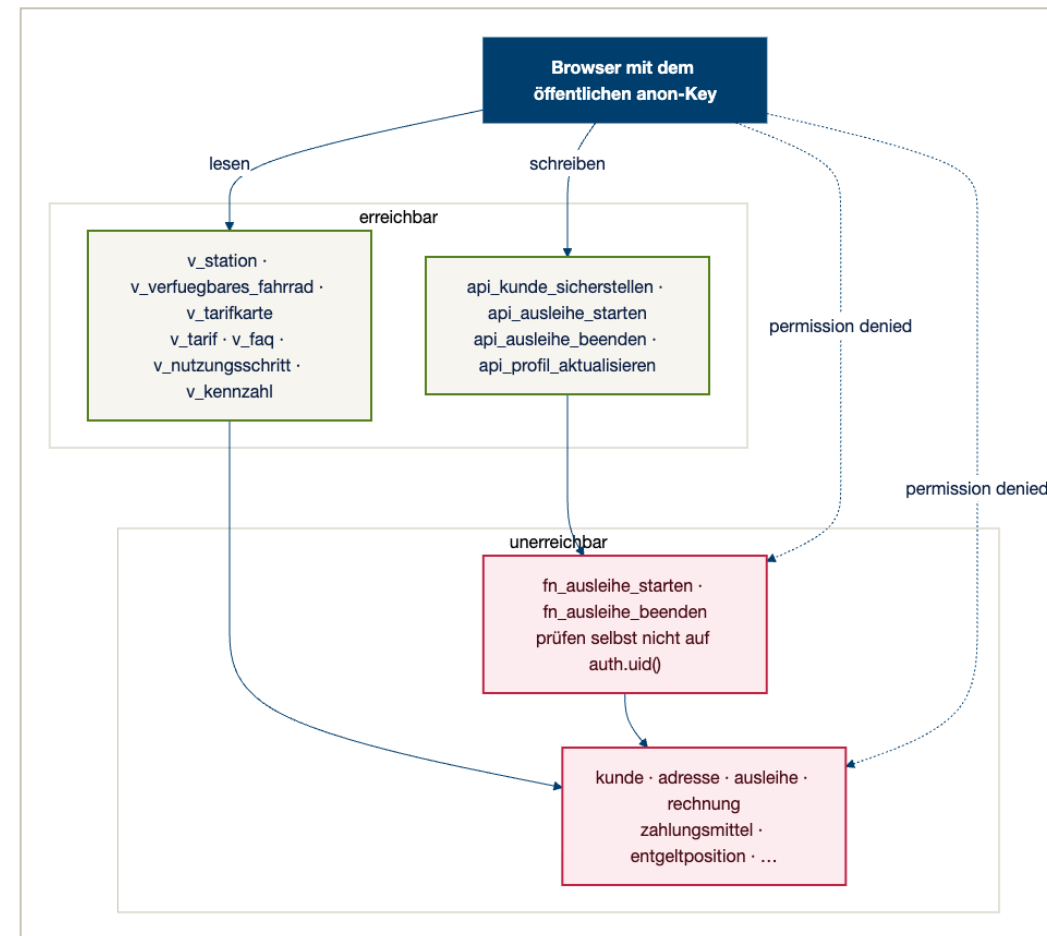
Anna sieht ihre Fahrt, sonst niemand.

Jede Zugriffsbeschränkung, die im JavaScript steht, ist wirkungslos.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Was der Browser erreicht — und was nicht

Zwei Schichten trennen die Anwendung von den Tabellen — die Fachlogik dazwischen prüft selbst nicht auf den angemeldeten Nutzer.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Die Falle, die fast jeder übersieht

PostgreSQL vergibt das Ausführungsrecht auf jede neu angelegte Funktion automatisch an die Rolle PUBLIC. Ein Entzug gegen einzelne Rollen greift deshalb nicht.

wirkungslos

REVOKE gegen die Rollen allein

```
revoke all on all functions  
in schema velocity  
from anon, authenticated;
```

Beide erben das Recht weiterhin
über PUBLIC. Die Fachlogik bleibt
aufrufbar.

wirksam

PUBLIC ausdrücklich mit entziehen

```
revoke all on all functions  
in schema velocity  
from public, anon, authenticated;
```

```
alter default privileges  
in schema velocity  
revoke execute on functions  
from public;
```

Aufgedeckt hat das ein Test, nicht Nachdenken. Rechte werden geprüft, nicht angenommen.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

Nachweis statt Behauptung — auf drei Wegen

Ein Sicherheitskonzept, das nur beschrieben ist, ist wertlos. Geprüft wird in der Datenbank, über die Schnittstelle und im Browser.

1 In der Datenbank RLS überall aktiv, anon ohne Tabellenrechte, Rollenwechsel-Probe

pgTAP, t0011

2 Über die Schnittstelle 13 gesperrte Ressourcen, 7 öffentliche Sichten

HTTP 401 gegen 200

3 Im Browser Aufrufe der abgemeldeten Seite in der Konsole

permission denied

Eine Falle im Prüfwerkzeug selbst

Der erste Entwurf meldete alle 13 gesperrten Ressourcen als bestanden — obwohl er nichts geprüft hatte: das Schema war gar nicht exponiert, also war alles unerreichbar, Sicheres wie Unsicheres.

Ein Test, der „abgesichert“ nicht von „gar nicht erreichbar“ unterscheidet, ist gefährlicher als kein Test.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

8 · Anwendung anbinden

Leitfrage

Wie kommt Annas Fahrt in die Web-Anwendung?

Nur Sichten lesen, nur Funktionen schreiben

Die Anwendung kennt keine einzige Basistabelle. Das ist keine Konvention, sondern erzwungen: sie käme gar nicht an sie heran.

Lesen — ausschließlich Sichten

```
from('v_station')
from('v_verfuegbares_fahrrad')
from('v_tarifkarte')
from('v_faq')
from('v_kennzahl')

-- angemeldet:
from('v_meine_ausleihe')
from('v_mein_profil')
```

Schreiben — ausschließlich Funktionen

```
rpc('api_kunde_sicherstellen')
rpc('api_ausleihe_starten')
rpc('api_ausleihe_beenden')
rpc('api_profil_aktualisieren')

-- nicht erreichbar:
from('kunde')      permission denied
from('ausleihe')   permission denied
rpc('fn_...')      permission denied
```

Annas Fahrt: Anna sieht ihre Fahrt über v_meine_ausleihe — begrenzt durch RLS auf ihre eigenen Zeilen.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.

9 · Zusammenfassung und Ausblick

Leitfrage

Was fehlt noch, damit Annas Rad auch gewartet wird?

Sechs Sätze, die diese Einheit tragen

Wenn Sie nichts anderes mitnehmen, dann diese sechs. Zu jedem sollten Sie ein Beispiel aus Annas Fahrt nennen können.

1 Reihenfolge Anforderungen vor Modell, Modell vor Relationen, Relationen vor DDL

2 Kardinalität ist eine fachliche Aussage, keine technische Festlegung

3 Normalform ist notwendig, aber nicht hinreichend

4 Constraint was nicht erzwungen wird, wird verletzt

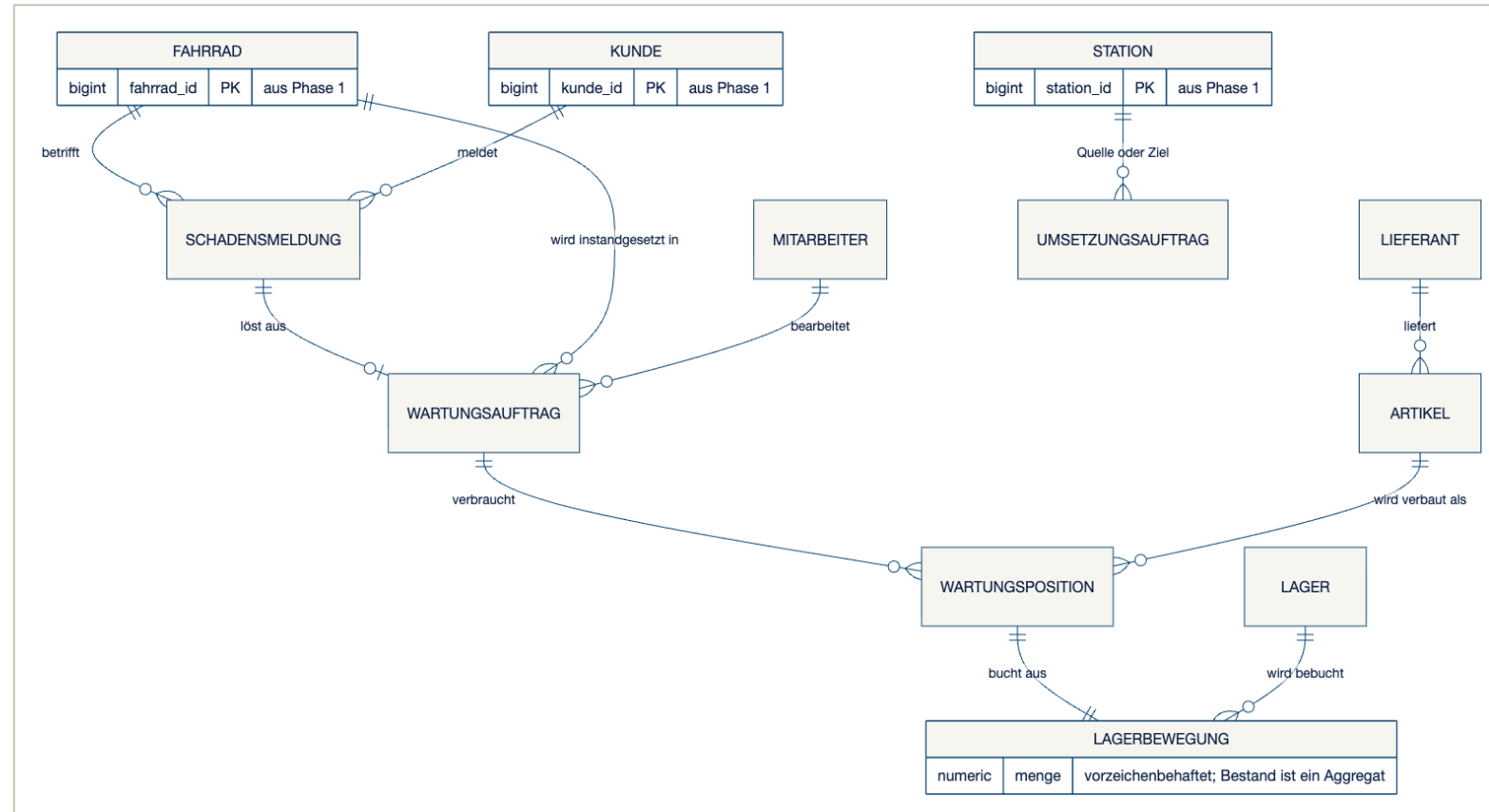
5 Schutz liegt in der Datenbank, nie im Browser

6 Nachweis Rechte werden geprüft, nicht angenommen

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter [doku/datenmodell/](#).

Die Warenwirtschaft hängt an denselben Entitäten

Vier weitere Fachbereiche sind bereits entworfen — deshalb war es richtig, hersteller und fahrradmodell schon jetzt einzuziehen.



Annas Fahrt: Meldet Anna einen Defekt, entsteht daraus ein Wartungsauftrag und eine Lagerbewegung.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen und Diagramme im Repository unter doku/datenmodell/.